



**05506026**

# **Digital Image Processing**

**Witchaya Towongpaichayont**  
**Computer Science, KMITL**

# CONTENTS

A decorative border on the left side of the slide composed of various overlapping triangles and polygons in shades of green, blue, red, and grey.

**1**

**IMAGE ALIGNMENT**

**2**

**IMAGE PATTERN CLASSIFICATION**

**3**

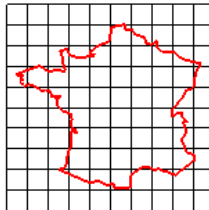
**WHAT ARE THE NEXT STEPS?**



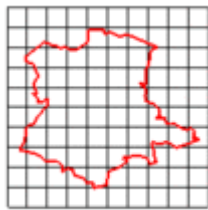
**IMAGE ALIGNMENT**

# Recap of transformations

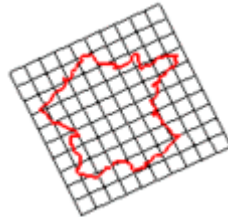
- ▶ In computer graphics, we learnt many geometric transformations including
  - ▶ Displacement (Translation) transformation
  - ▶ Rigid (Euclidean) transformation
  - ▶ Similarity transformation
  - ▶ Affine transformation
- ▶ These are used to transform objects while preserve some properties



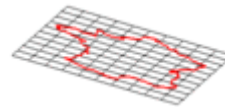
**Original**



**Euclidean**



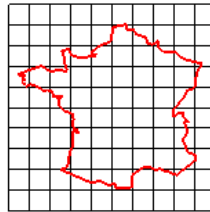
**Similarity**



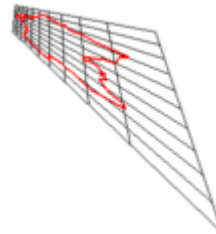
**Affine**

# Homography

- ▶ Homography, or projective transformation, is one of geometric transformations where it preserves collinearity (lines to lines)
- ▶ Homography is useful in digital image processing to map one image into another image using a 3x3 transformation matrix to maps points in one image to the corresponding points in another image



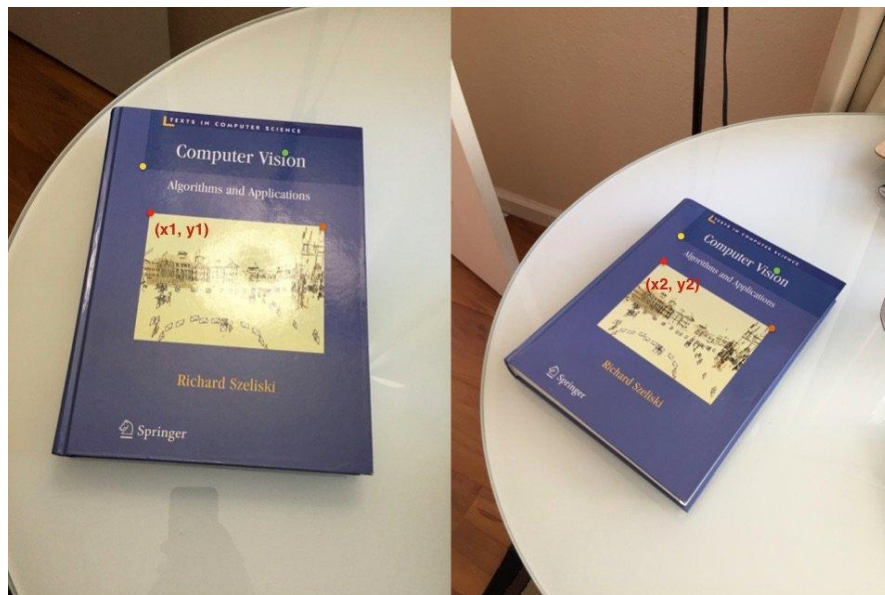
**Original**



**Homography**

# Homography

---



# Homography

- ▶ If two images contain the same surface, then there exists a 3x3 matrix  **$H$**  in which we can map point  **$p$**  into the corresponding point  $p'$  such that

$$p' = Hp$$

- ▶ We can write the 3x3 homography matrix as

$$H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix}$$

- ▶ Therefore,

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = H \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

# Homography

- ▶ If we are certain that two points are corresponding, we can use these points to find the value in matrix H
- ▶ From,

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- ▶ We can conclude that

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \frac{h_{11}x + h_{12}y + h_{13}}{h_{31}x + h_{32}y + h_{33}} \\ \frac{h_{21}x + h_{22}y + h_{23}}{h_{31}x + h_{32}y + h_{33}} \end{bmatrix}$$



# Homography

- ▶ Since homography is based on projective transformation, we can multiply all elements of  $H$  by a non-zero  $k$  without affecting the solution
- ▶ Therefore, we can divide all elements of  $H$  by  $h_{33}$
- ▶ Then

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \frac{h_{11}x + h_{12}y + h_{13}}{h_{31}x + h_{32}y + 1} \\ \frac{h_{21}x + h_{22}y + h_{23}}{h_{31}x + h_{32}y + 1} \end{bmatrix}$$

- ▶ (Note that it does not matter what we call these variables, so the other variables other than  $h_{33}$  remain the same)

# Homography

---

▷ From the matrix form  $\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \frac{h_{11}x+h_{12}y+h_{13}}{h_{31}x+h_{32}y+1} \\ \frac{h_{21}x+h_{22}y+h_{23}}{h_{31}x+h_{32}y+1} \end{bmatrix}$

▷ We can write in algebra form as

$$x' = \frac{h_{11}x + h_{12}y + h_{13}}{h_{31}x + h_{32}y + 1}$$
$$y' = \frac{h_{21}x + h_{22}y + h_{23}}{h_{31}x + h_{32}y + 1}$$

▷ Which can be reorganised as

$$x' = h_{11}x + h_{12}y + h_{13} - h_{31}xx' - h_{32}yx'$$
$$y' = h_{21}x + h_{22}y + h_{23} - h_{31}xy' - h_{32}yy'$$

# Homography

---

- ▶ The eight elements of Homography matrix can be calculated if there are enough points
- ▶ Typically, **4 points** are sufficient to be used to calculate the values in the matrix
- ▶ The question is, how do we know that which points in the image are corresponding?

# Maximally Stable Extremal Regions (MSERs)

- ▶ MSERs are interested in the stable region to establish correspondence between two or more images
- ▶ The idea of the algorithm is to engage the region of the pixels where the most stable when changing the threshold value for binarization

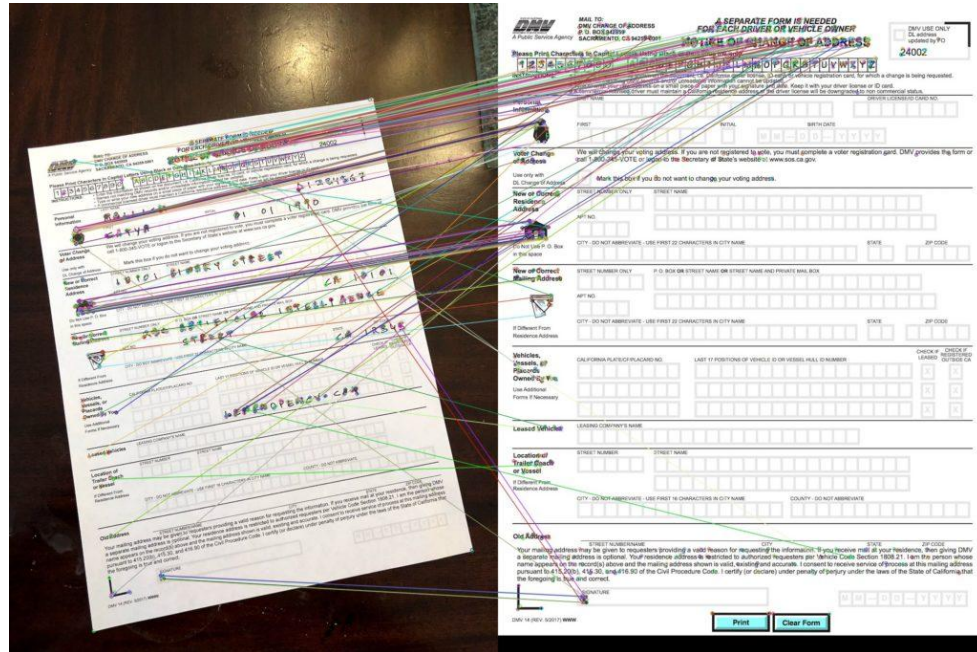


# Local Extrema (Keypoints)

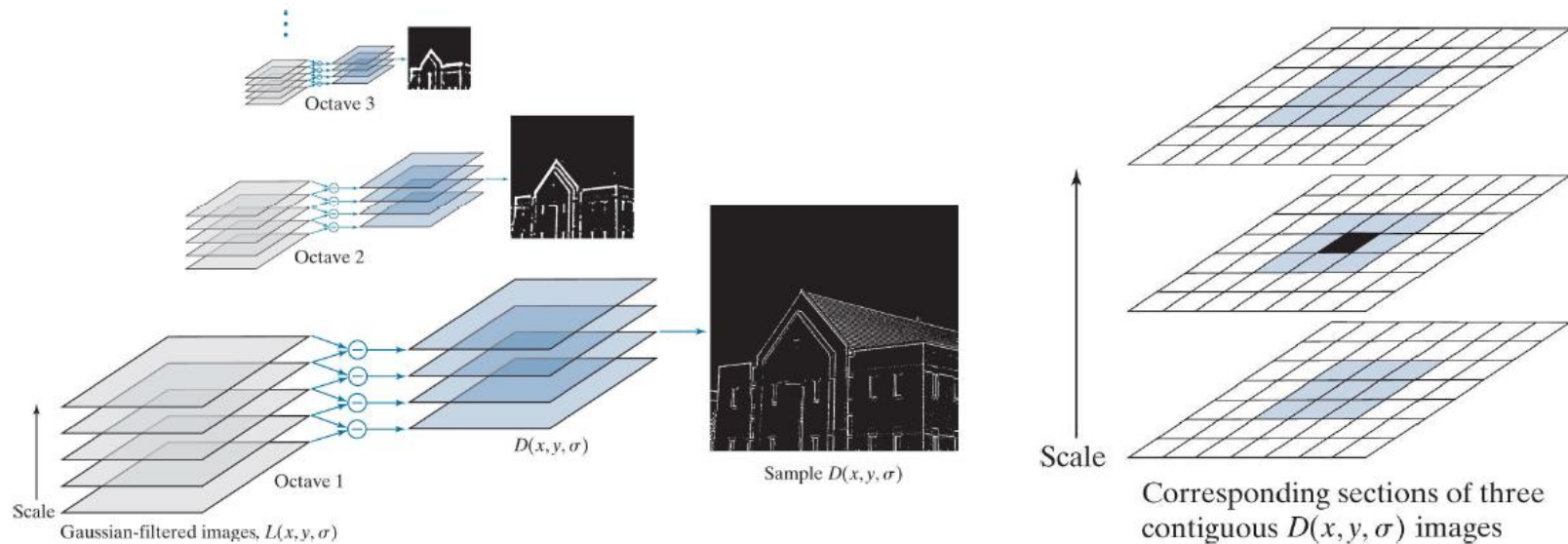
---

- ▶ Keypoints or feature points are the stable points in an image
- ▶ These keypoints are used a lot in computer vision
- ▶ The steps of using keypoints are
  - ▶ Detect
  - ▶ Describe
  - ▶ Match
- ▶ Detected keypoints with the same description in two images will be matched
- ▶ There are several algorithms such as SIFT, SURF, ORB, etc.

# Local Extrema (Keypoints)



# Keypoints detection using SIFT Algorithm



# Keypoints detection using SIFT Algorithm

---



Image from: Gonzalez, R., & Woods, R. (2014). Digital image processing (3rd ed.). New Delhi: Dorling Kindersley.



# Keypoints description (orientation) using SIFT Algorithm

---



Image from: Gonzalez, R., & Woods, R. (2014). Digital image processing (3rd ed.). New Delhi: Dorling Kindersley.

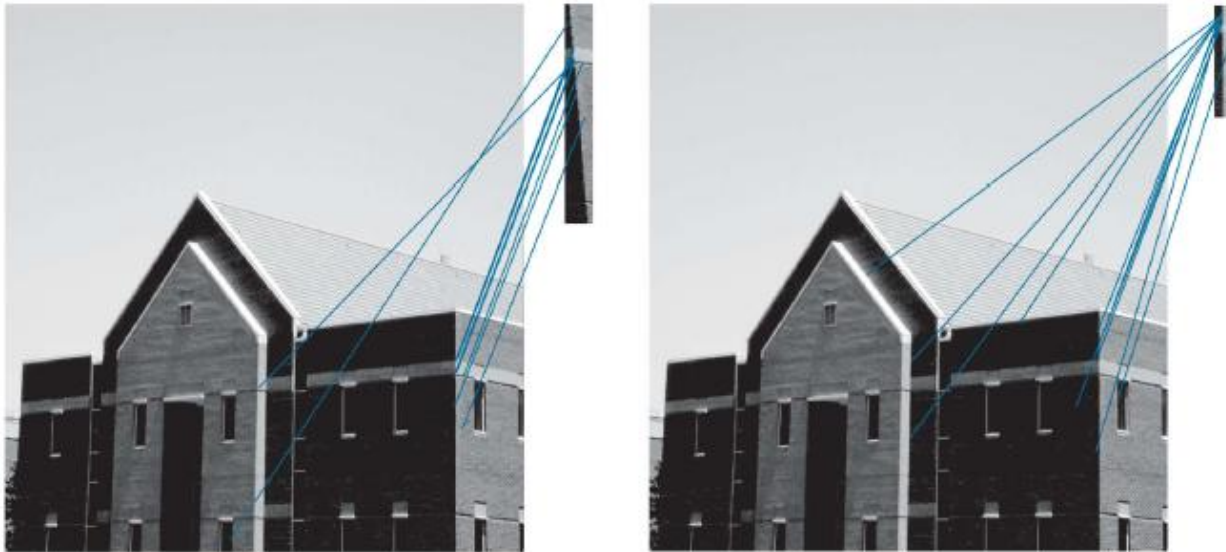
# Image Matching using SIFT Algorithm

---



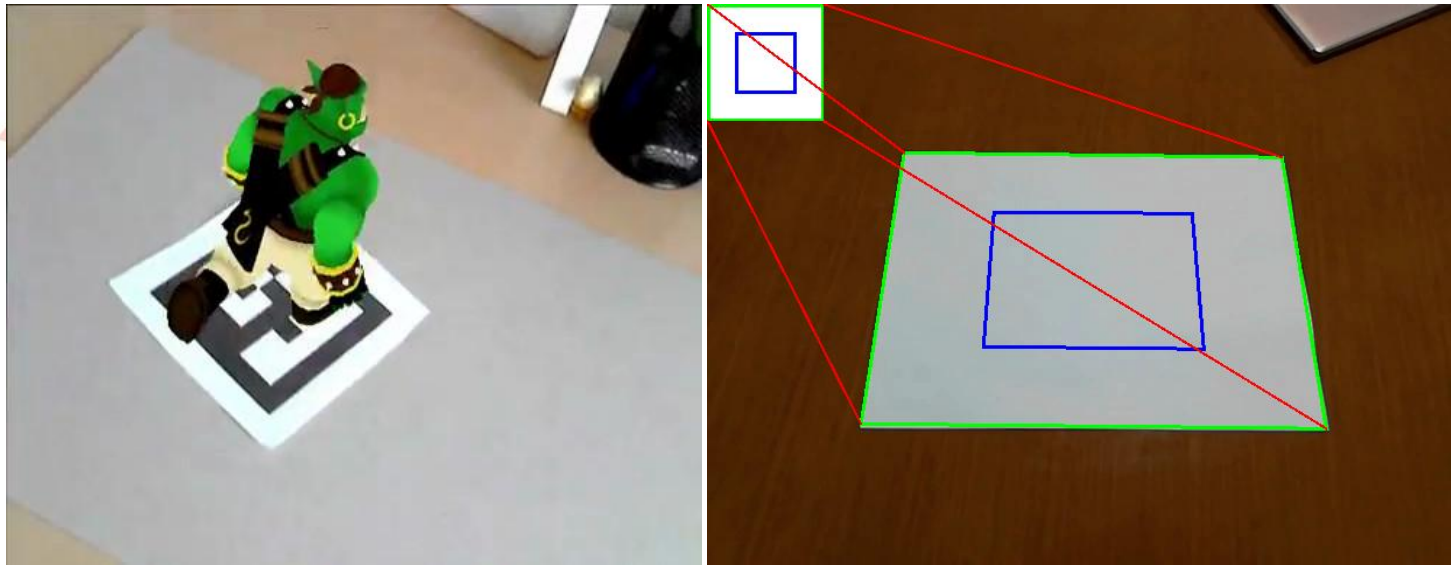
# Image Matching using SIFT Algorithm

---



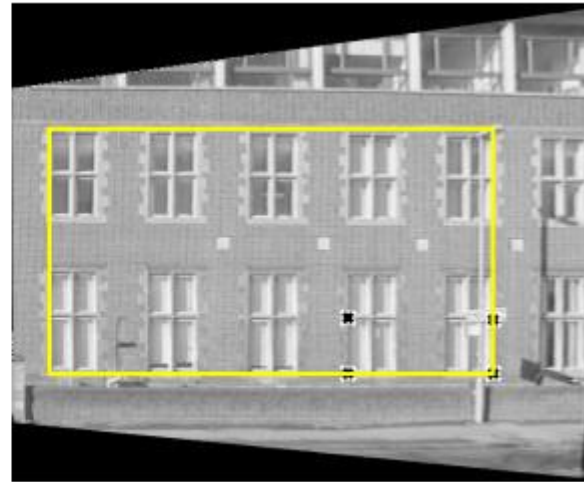
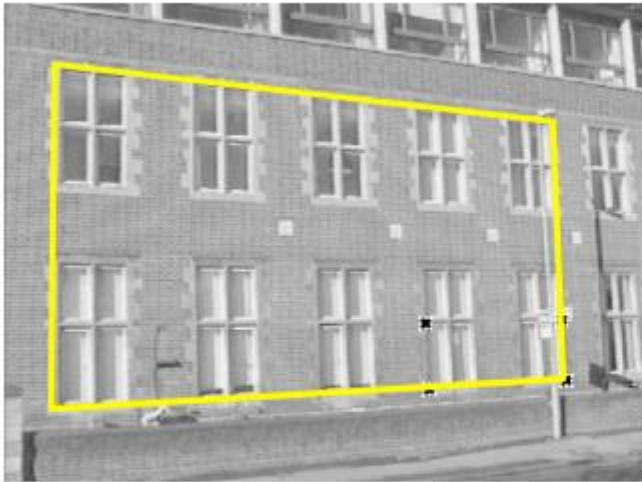
# Usages of image alignment

---



# Usages of image alignment

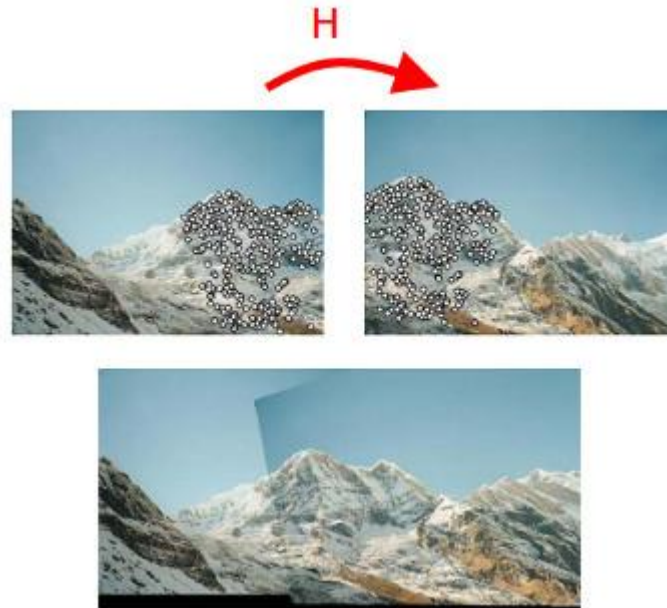
---



from Hartley & Zisserman

# Usages of image alignment

---







# IMAGE PATTERN CLASSIFICATION

# Background

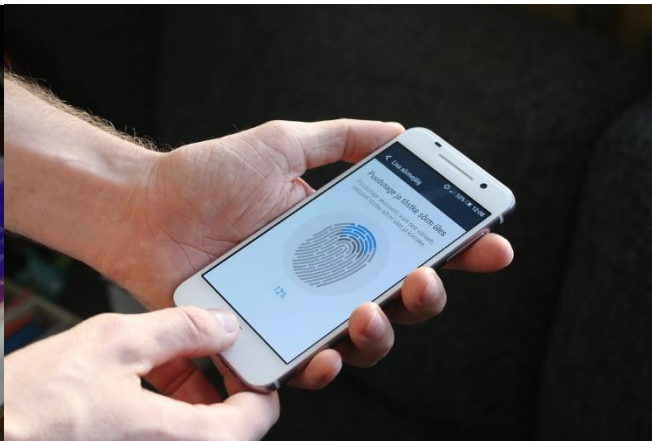
---

- ▶ Pattern recognition is one ability which humans possess the best in the known biological world
- ▶ Machine, on the other hand, does not have such great ability
- ▶ However, pattern recognitions have important roles in human daily life



# Background

---



# Background

---

- ▷ **Pattern** of image is a spatial arrangement of features
- ▷ **Pattern class** is a set of patterns that share some common properties
- ▷ **Pattern recognition** is a set of techniques which automatically assign patterns to their respective class

# Background

---

- ▶ Four main stages in image pattern recognition is
  1. Sensing
  2. Preprocessing
  3. Feature Extraction
  4. Classification

We have learnt stage 1 – 3 already

# Background

---

- ▶ Image pattern classification can be categorised into three principles
  - ▶ Prototype matching
  - ▶ Optimal statistical formulation
  - ▶ Neural networks
- ▶ The first two approaches are used when the nature of the data is well understood, while the latter one requires to select pattern class characteristics well enough for the system to learn

# Background

---

- ▶ But one characteristic shared by preceding three approaches is that they are based on parameters that must be either specified or learned from patterns that represent the recognition problem we want to solve
- ▶ The patterns can be ***labeled***, meaning that we know the class of each pattern, or ***unlabeled***, meaning that the data are known to be patterns, but the class of each pattern is unknown

# Background

---

- ▶ When working with a labeled data, a given data set generally is subdivided into three subsets: a training set, a validation set, and a test set (a typical subdivision might be 50% training, and 25% each for the validation and test sets)
  - ▶ Training set is images with label to adjust the parameters of the classifier
  - ▶ Validation set is images to compare the various designs against a performance objective
  - ▶ Test set is images with patterns that the system has never seen before



## Patterns and Pattern Classes

# Patterns and Pattern Classes

---

- ▶ Image pattern classification has two arrangements of principle: - quantitative and structural
  - ▶ Quantitative patterns are in the form of ***pattern vectors***
  - ▶ Structural patterns are composed of ***symbols, strings, trees, or graphs***



# Pattern Vector

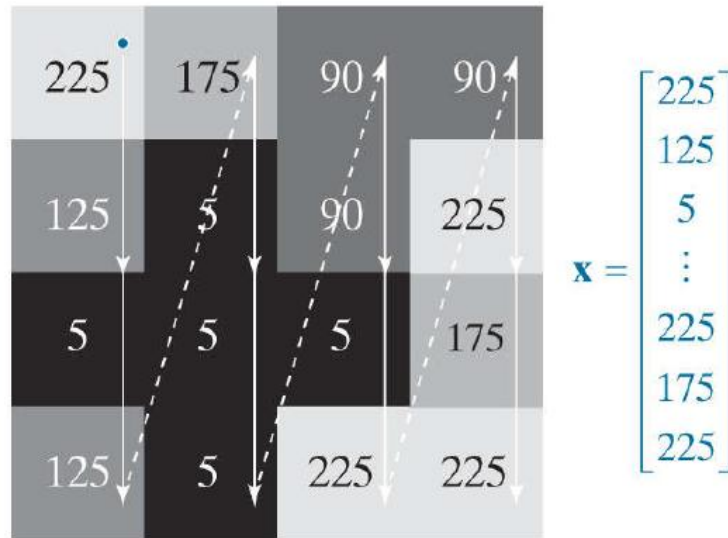
- ▶ Pattern vectors are represented by lower letters such as  $x$ ,  $y$ ,  $z$ , in the form

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

- ▶ Where each component,  $x_i$ , represents the  $i$ -th feature descriptor and  $n$  is the total number of descriptors
- ▶ The equivalent row form is  $\mathbf{x} = (x_1, x_2, \dots, x_n)^T$  where  $T$  indicates transposition

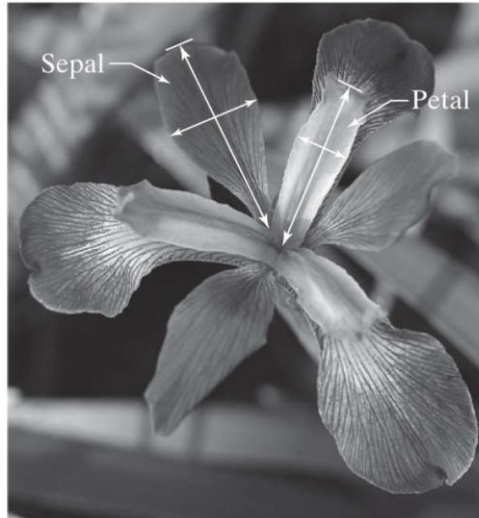
# Pattern Vector

- ▶ One of pattern vector examples is the intensities of a region in the form of linear indexing



# Pattern Vector

- ▶ Another pattern vector example, which is in a higher level, is based on feature descriptors in the previous week.



$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}$$

$x_1$  = Petal width

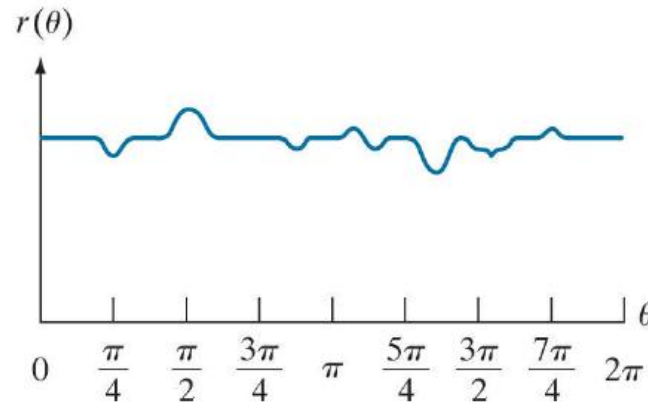
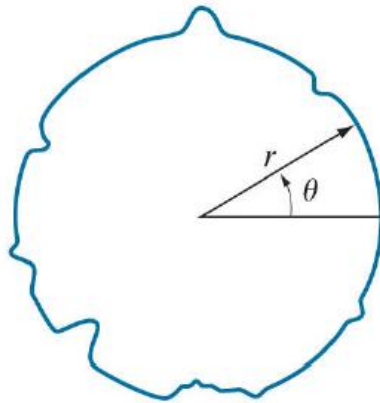
$x_2$  = Petal length

$x_3$  = Sepal width

$x_4$  = Sepal length

# Pattern Vector

---



$$\mathbf{x} = \begin{bmatrix} g(r(\theta_1)) \\ g(r(\theta_2)) \\ \vdots \\ g(r(\theta_n)) \end{bmatrix}$$

# Pattern Vector

---



$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

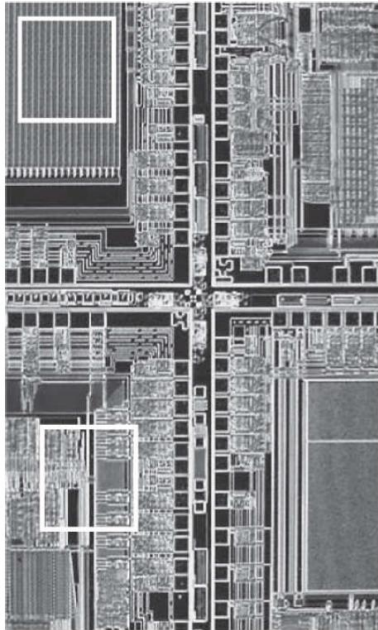
$x_1$  = compactness

$x_2$  = circularity

$x_3$  = eccentricity

# Pattern Vector

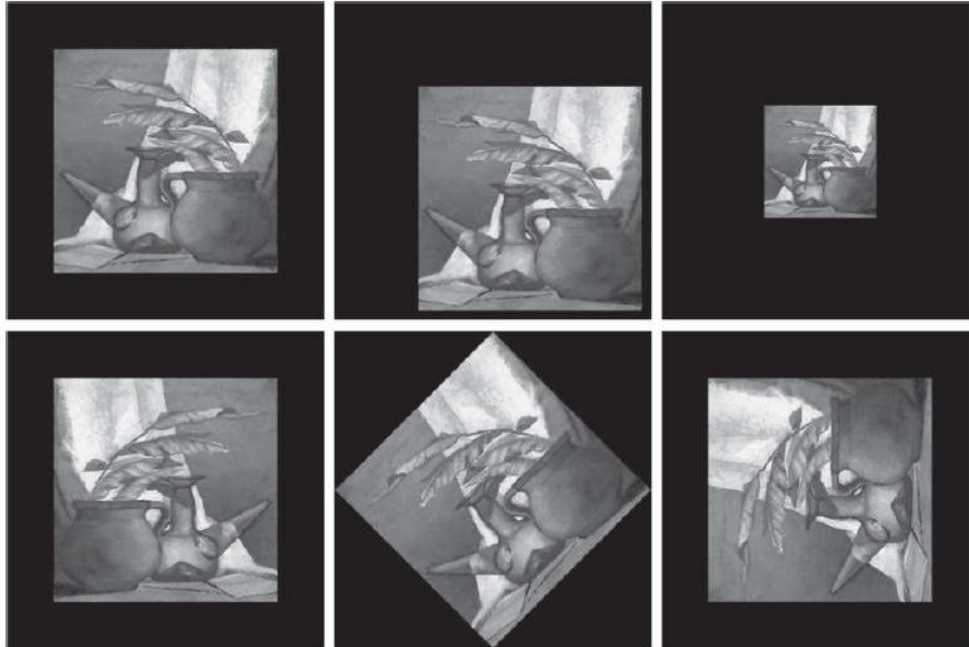
---



$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \end{bmatrix}$$

$x_1$  = max probability  
 $x_2$  = correlation  
 $x_3$  = contrast  
 $x_4$  = uniformity  
 $x_5$  = homogeneity  
 $x_6$  = entropy

# Pattern Vector

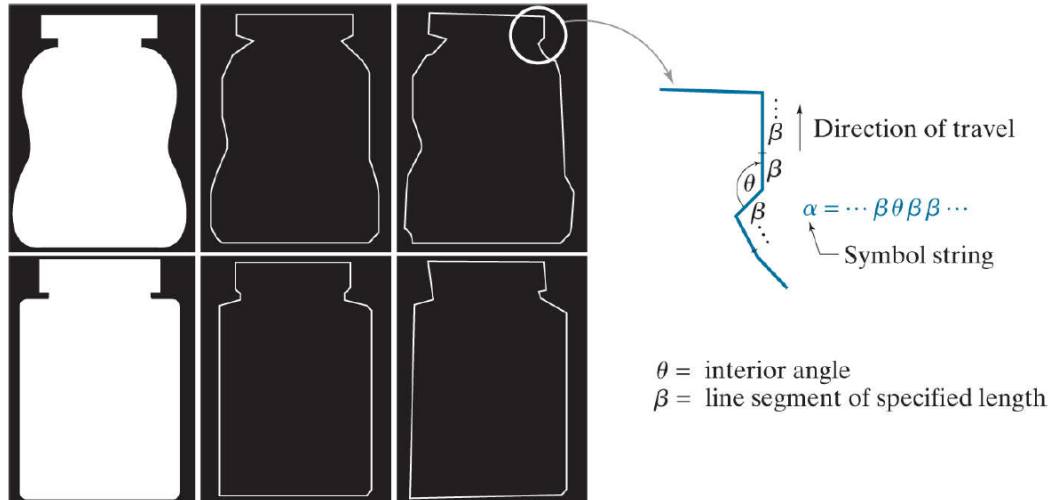


$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \end{bmatrix} = \begin{bmatrix} \phi_1 \\ \phi_2 \\ \phi_3 \\ \phi_4 \\ \phi_5 \\ \phi_6 \\ \phi_7 \end{bmatrix}$$

The  $\phi$ 's are moment invariants

# Structural Patterns

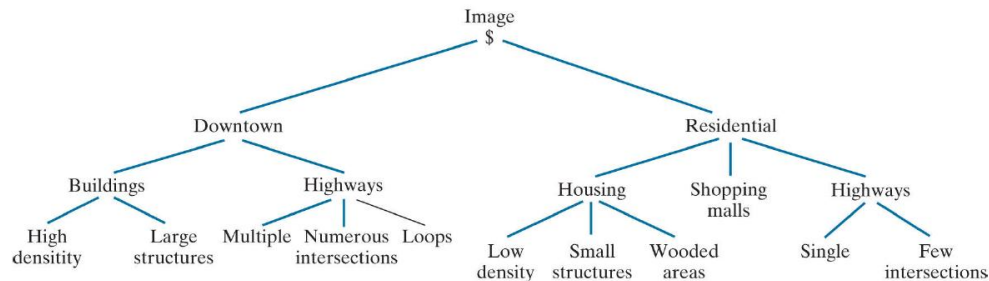
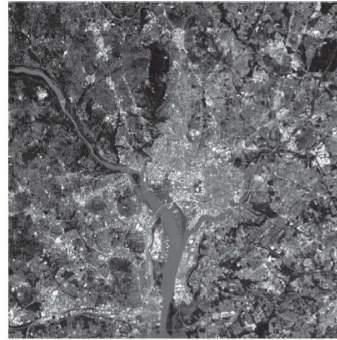
- ▶ Structural patterns are suitable for objects which are represented by structural features, such as strings of symbols





# Structural Patterns

## ► Or tree





## **Classification based on prototype matching**

# Prototype Matching

- 
- ▶ Prototype matching involves comparing an unknown pattern against a set of prototypes, and assigning to the class that is the most similar

# Minimum-Distance Classifier

- ▶ Also referred to as the nearest-neighbour classifier.
- ▶ It is one of the simplest and most widely used methods.
- ▶ It computes a distance-based measure between the unknown pattern vector and the class prototypes. The number to be measured is usually the mean vectors of various pattern classes

$$\mathbf{m}_j = \frac{1}{n_j} \sum_{\mathbf{x} \in c_j} \mathbf{x} \quad j = 1, 2, \dots, N_c$$

- ▶ Then the distance is the Euclidean distance of two vectors

$$D_j(\mathbf{x}) = \|\mathbf{x} - \mathbf{m}_j\| \quad j = 1, 2, \dots, N_c$$

- ▶ Where  $\|\mathbf{a}\| = (\mathbf{a}^T \mathbf{a})^{1/2}$  is the Euclidean norm

# Minimum-Distance Classifier

- ▶ From the Euclidean norm  $\|\mathbf{a}\| = (\mathbf{a}^T \mathbf{a})^{1/2}$ , the smallest distance is equivalent to the function

$$d_j(\mathbf{x}) = \mathbf{m}_j^T \mathbf{x} - \frac{1}{2} \mathbf{m}_j^T \mathbf{m}_j \quad j = 1, 2, \dots, N_c$$

- ▶ The decision boundary between two classes is the boundary where

$$d_i(\mathbf{x}) = d_j(\mathbf{x})$$

- ▶ Which then we can calculate the decision boundaries at

$$\begin{aligned} d_{ij}(\mathbf{x}) &= d_i(\mathbf{x}) - d_j(\mathbf{x}) \\ &= (\mathbf{m}_i - \mathbf{m}_j)^T \mathbf{x} - \frac{1}{2} (\mathbf{m}_i - \mathbf{m}_j)^T (\mathbf{m}_i + \mathbf{m}_j) = 0 \end{aligned}$$

# Minimum-Distance Classifier

- ▶ For example, Iris flower has different species which can be separated using the width and height of the sepals and petals



# Minimum-Distance Classifier

- ▶ We can denote the Iris versicolor and setosa data as classes  $c_1$  and  $c_2$  respectively
- ▶ The means of two classes are  $\mathbf{m}_1 = (4.3, 1.3)^T$  and  $\mathbf{m}_2 = (1.5, 0.3)^T$  which we can calculate the smallest distance as

$$\begin{aligned}d_1(x) &= m_1^T x - \frac{1}{2} m_1^T m_1 \\&= 4.3x_1 + 1.3x_2 - \frac{1}{2}(4.3^2 + 1.3^2) \\&= 4.3x_1 + 1.3x_2 - 10.1\end{aligned}$$

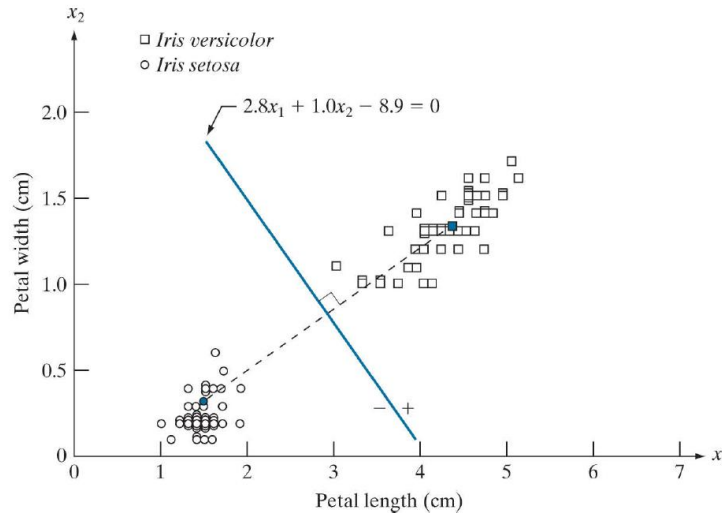
And

$$\begin{aligned}d_2(x) &= m_2^T x - \frac{1}{2} m_2^T m_2 \\&= 1.5x_1 + 0.3x_2 - \frac{1}{2}(1.5^2 + 0.3^2) \\&= 1.5x_1 + 0.3x_2 - 1.17\end{aligned}$$

# Minimum-Distance Classifier

- ▶ Which means the equation of the boundary is

$$\begin{aligned}d_{12}(x) &= d_1(x) - d_2(x) \\ &= 2.8x_1 + 1.0x_2 - 8.9 = 0\end{aligned}$$





# Minimum-Distance Classifier

- ▶ Another example is E-13B font which is used widely since middle 1950s to automate the check processing. The font set consists of 14 characters on a 9 x 7 grid. The font became the standard by American Banker's Association

**維基銀行** Wiki Bank 樣本 Sample

---

The Wikipedia Banking Corporation Limited 維基銀行(香港)有限公司  
Hong Kong SAR 香港特別行政區

|     |       |      |
|-----|-------|------|
| 31  | 02    | 9999 |
| Day | Month | Year |

Pay 新付 甲乙丙丁有限公司 ABCD Co. Ltd. or bearer 或持票人

---

HK dollars 港幣 玖萬捌千柒佰陸拾伍元正 HK \$ 98,765.00

Ninety-eight Thousand Seven Hundred and Sixty-five Dollars Only

MR CHAN MAN



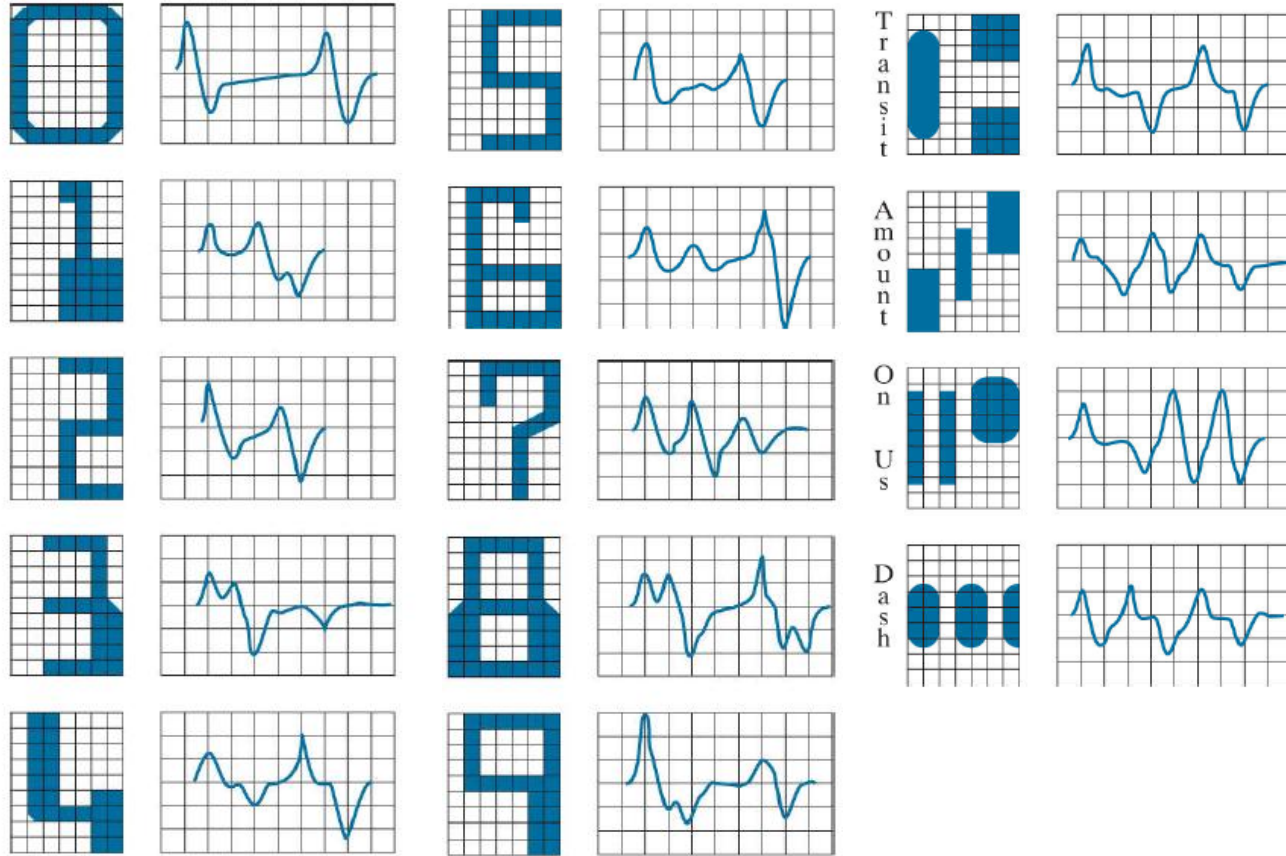
Signed after the Payment has been made and the receipt has been received by the bank.  
Signed after the Payment has been made and the receipt has been received by the bank.

By Phantomkid © Wikipedia.Zh

||.. 2 3 4 5 6 ||.. 9 8 7 6 5 4 : 0 9 8 7 6 5 0 0 0 ||..

1 2 3 4 5 6 7 8 9 0 : ; ' " , . ' " ...

- The characters are stylised to maximise the difference between them while maintain the readability for humans



# Using Correlation for 2-D Prototype Matching

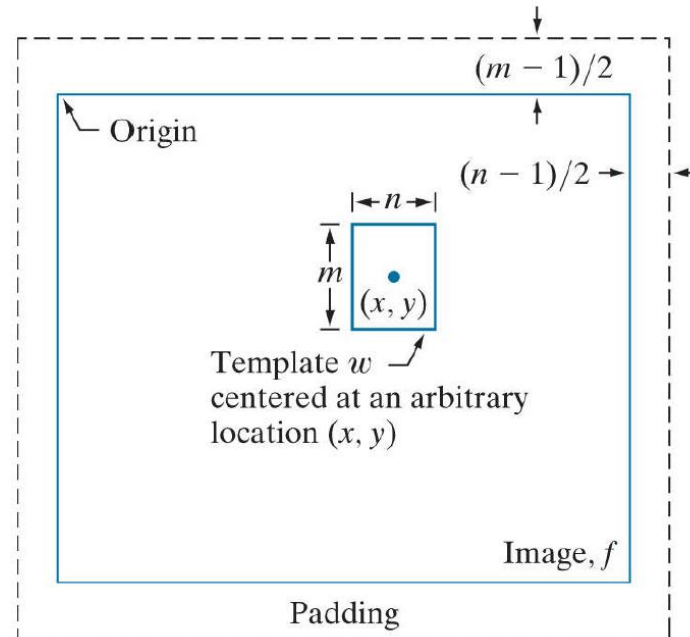
- ▶ As we know, the correlation of a kernel  $w$  with an image  $f(x, y)$  is

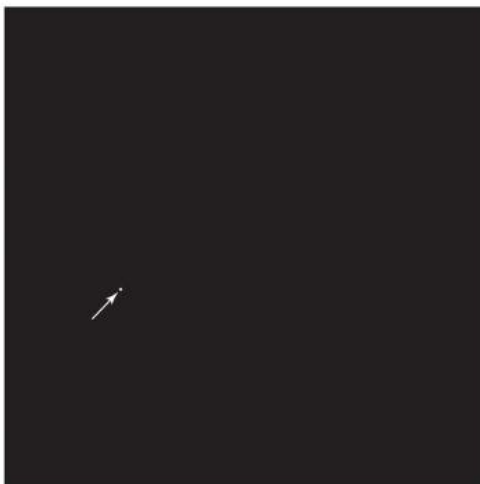
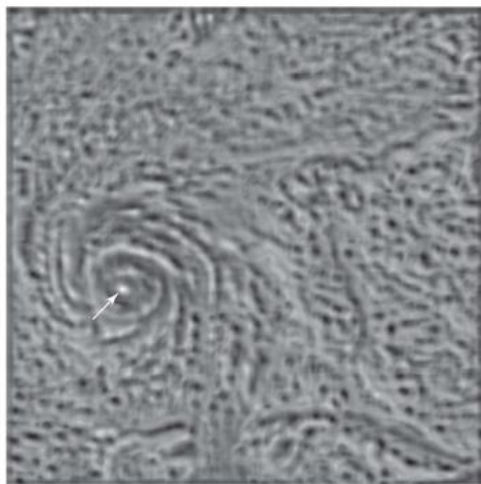
$$(w \star f)(x, y) = \sum_s \sum_t w(s, t) f(x + s, y + t)$$

- ▶ The correlation will give the highest value in the region where  $f$  and  $w$  are equal or nearly equal. This means we can use this property to find the locations where  $w$  matches a region of  $f$
- ▶ In order to use it, we must normalise correlation to avoid the sensitivity to changes in amplitude of either function by using the correlation coefficient instead

$$\gamma(x, y) = \frac{\sum_s \sum_t [w(s, t) - \bar{w}][f(x + s, y + t) - \bar{f}_{xy}]}{\left\{ \sum_s \sum_t [w(s, t) - \bar{w}]^2 \sum_s \sum_t [f(x + s, y + t) - \bar{f}_{xy}]^2 \right\}^{\frac{1}{2}}}$$

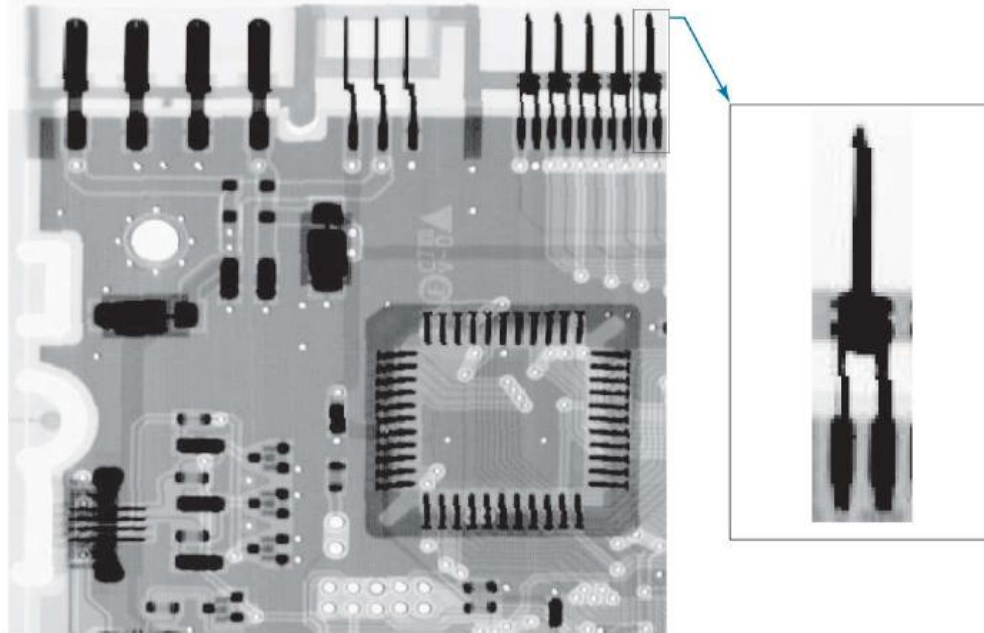
# Using Correlation for 2-D Prototype Matching





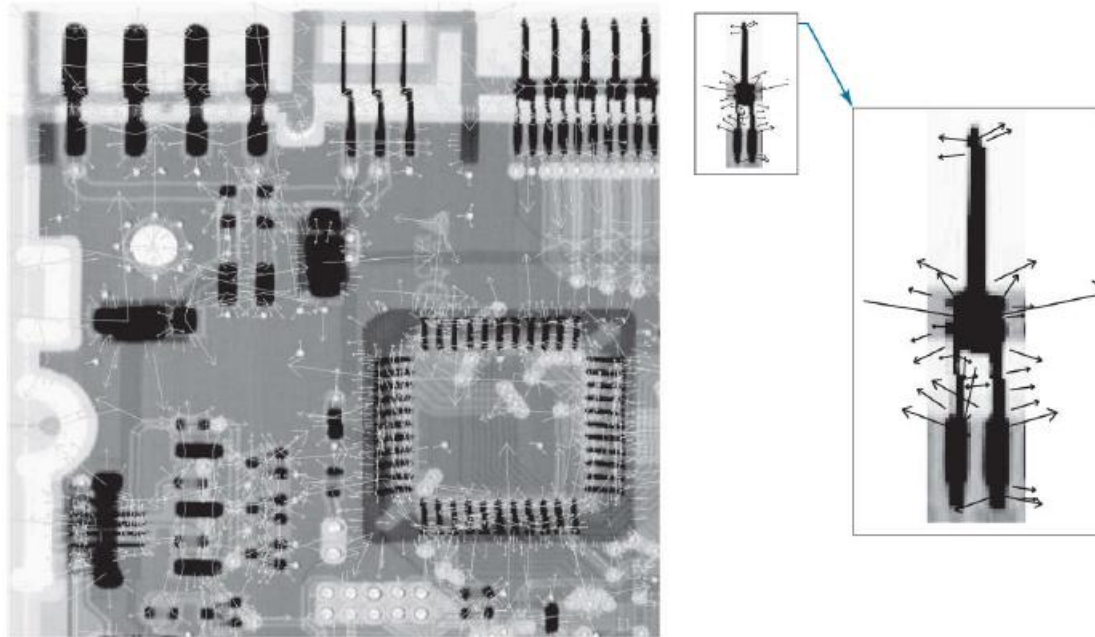
# Matching SIFT Features

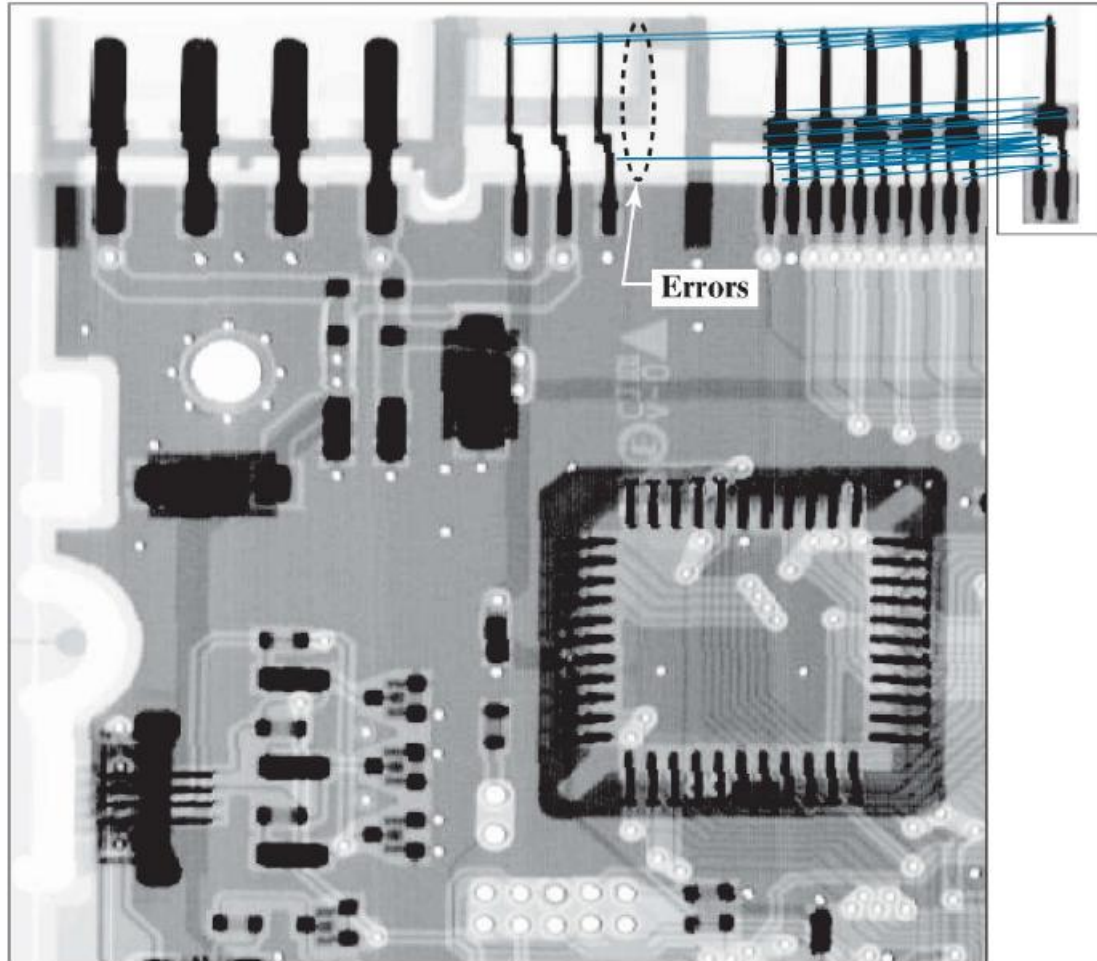
---





# Matching SIFT Features

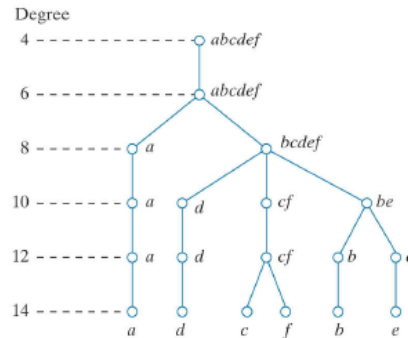
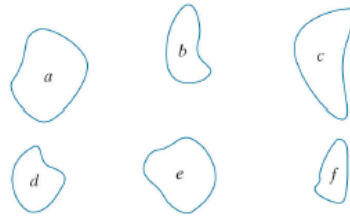






# Matching Structural Prototypes

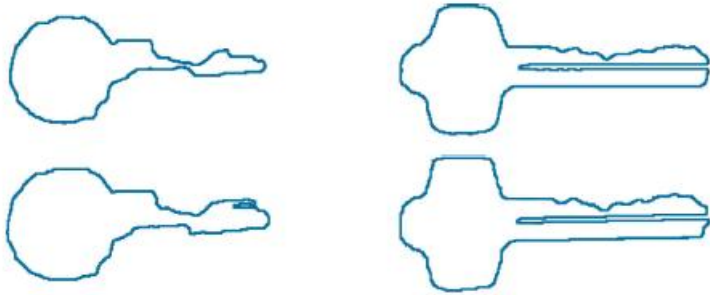
## ▶ Matching shape numbers



|          | <i>a</i> | <i>b</i> | <i>c</i> | <i>d</i> | <i>e</i> | <i>f</i> |
|----------|----------|----------|----------|----------|----------|----------|
| <i>a</i> | $\infty$ | 6        | 6        | 6        | 6        | 6        |
| <i>b</i> |          | $\infty$ | 8        | 8        | 10       | 8        |
| <i>c</i> |          |          | $\infty$ | 8        | 8        | 12       |
| <i>d</i> |          |          |          | $\infty$ | 8        | 8        |
| <i>e</i> |          |          |          |          | $\infty$ | 8        |
| <i>f</i> |          |          |          |          |          | $\infty$ |

# Matching Structural Prototypes

## ► String matching



| R   | 1.a  | 1.b  | 1.c  | 1.d  | 1.e  | 1.f | R   | 2.a  | 2.b | 2.c  | 2.d  | 2.e  | 2.f |
|-----|------|------|------|------|------|-----|-----|------|-----|------|------|------|-----|
| 1.a | ∞    |      |      |      |      |     | 2.a | ∞    |     |      |      |      |     |
| 1.b | 16.0 | ∞    |      |      |      |     | 2.b | 33.5 | ∞   |      |      |      |     |
| 1.c | 9.6  | 26.3 | ∞    |      |      |     | 2.c | 4.8  | 5.8 | ∞    |      |      |     |
| 1.d | 5.1  | 8.1  | 10.3 | ∞    |      |     | 2.d | 3.6  | 4.2 | 19.3 | ∞    |      |     |
| 1.e | 4.7  | 7.2  | 10.3 | 14.2 | ∞    |     | 2.e | 2.8  | 3.3 | 9.2  | 18.3 | ∞    |     |
| 1.f | 4.7  | 7.2  | 10.3 | 8.4  | 23.7 | ∞   | 2.f | 2.6  | 3.0 | 7.7  | 13.5 | 27.0 | ∞   |

| R   | 1.a  | 1.b  | 1.c  | 1.d  | 1.e  | 1.f  |
|-----|------|------|------|------|------|------|
| 2.a | 1.24 | 1.50 | 1.32 | 1.47 | 1.55 | 1.48 |
| 2.b | 1.18 | 1.43 | 1.32 | 1.47 | 1.55 | 1.48 |
| 2.c | 1.02 | 1.18 | 1.19 | 1.32 | 1.39 | 1.48 |
| 2.d | 1.02 | 1.18 | 1.19 | 1.32 | 1.29 | 1.40 |
| 2.e | 0.93 | 1.07 | 1.08 | 1.19 | 1.24 | 1.25 |
| 2.f | 0.89 | 1.02 | 1.02 | 1.24 | 1.22 | 1.18 |



**Classification based on optimal  
statistical formulation**

# Derivation of the Bayes Classifier

---

- ▶ Using optimal statistics mean that we will measure the probability of committing classification errors, in the other word, giving the lowest probability
- ▶ The Bayes classifier is a 0-1 loss function computes in the form

$$d_j(\mathbf{x}) = p(\mathbf{x}/c_j)P(c_j) \quad j = 1, 2, \dots, N_c$$

# Derivation of the Bayes Classifier

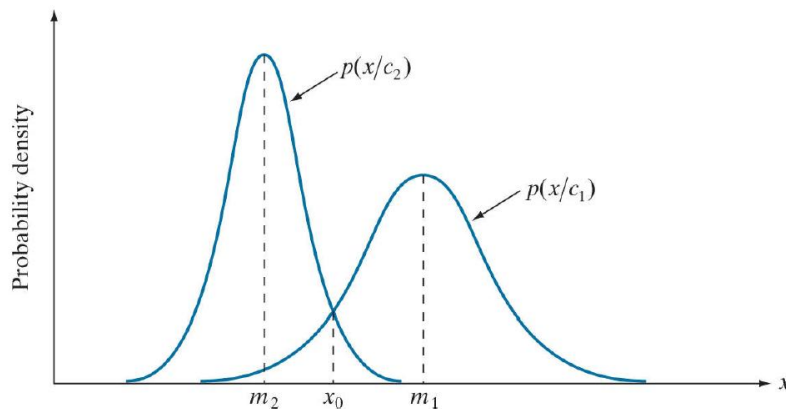
---

- For example, in a 1-D space where we have two Gaussian-density pattern classes, with means  $m_1$  and  $m_2$ , and standard deviations  $\sigma_1$  and  $\sigma_2$  respectively. The Bayes decision functions have the form

$$\begin{aligned} d_j(x) &= p(x/c_j)P(c_j) \\ &= \frac{1}{\sqrt{2\pi}\sigma_j} e^{-\frac{(x-m_j)^2}{2\sigma_j^2}} P(c_j) \quad j = 1, 2 \end{aligned}$$

# Derivation of the Bayes Classifier

- ▶ There is a single point  $x_0$  such that  $d_1(x_0) = d_2(x_0)$ . Since the two classes are equally likely to occur, then  $P(c_1) = P(c_2) = \frac{1}{2}$



- ▶ Therefore, solving the equation of Bayes decision form in which the probability is  $\frac{1}{2}$  will give  $x_0$ . The points on the left of  $x_0$  belong to  $c_2$  and the points on the right belong to  $c_1$

# Derivation of the Bayes Classifier

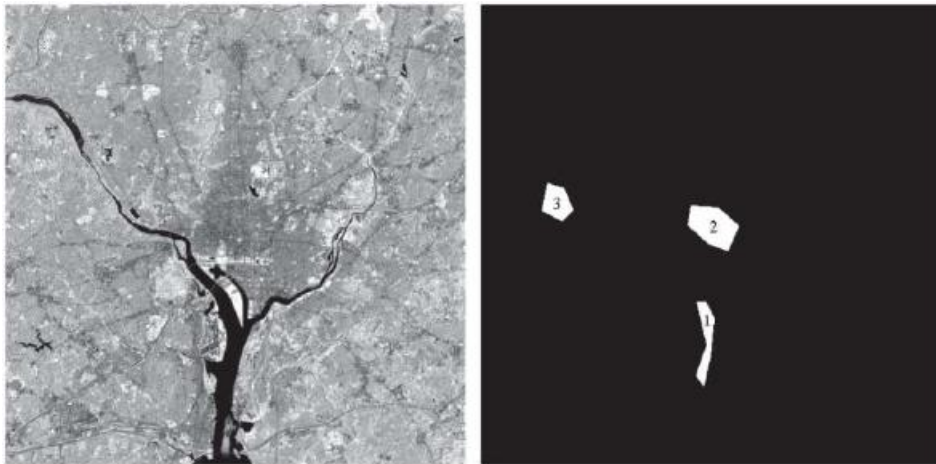
---

- ▶ In the n-dimensional case, the Gaussian density of the vectors in the jth pattern class has the form

$$p(\mathbf{x}/c_j) = \frac{1}{(2\pi)^{n/2} |\mathbf{C}_j|^{1/2}} e^{-\frac{1}{2} (\mathbf{x} - \mathbf{m}_j)^T \mathbf{C}_j^{-1} (\mathbf{x} - \mathbf{m}_j)}$$

# Derivation of the Bayes Classifier

---





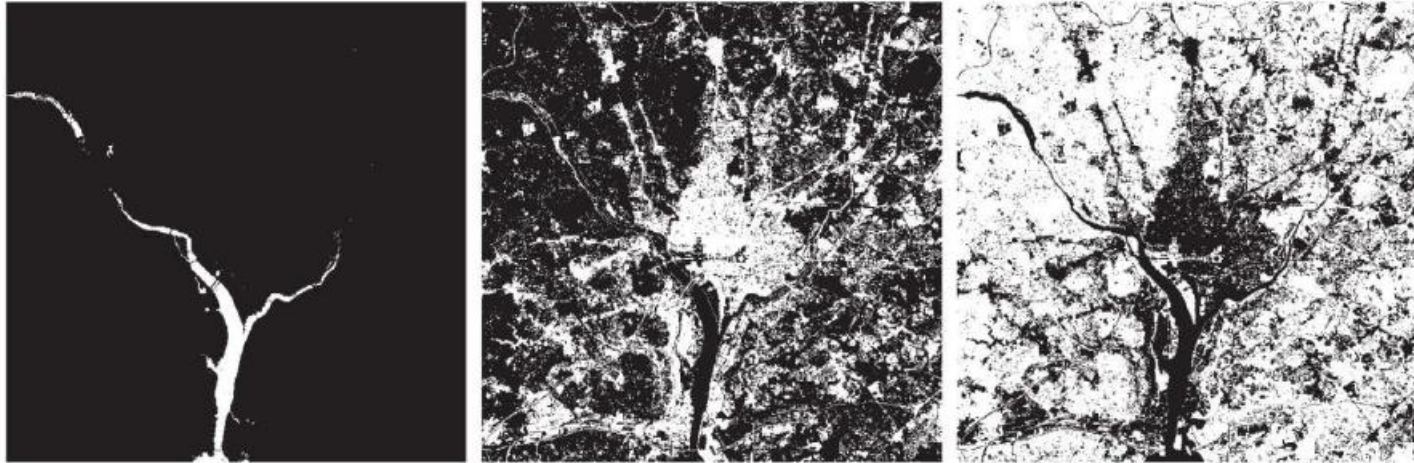
# Derivation of the Bayes Classifier

Bayes classification of multispectral image data. Classes 1, 2, and 3 are water, urban, and vegetation, respectively.

| Training Patterns |                |                       |     |     |           | Test Patterns |                |                       |     |     |           |
|-------------------|----------------|-----------------------|-----|-----|-----------|---------------|----------------|-----------------------|-----|-----|-----------|
| Class             | No. of Samples | Classified into Class |     |     | % Correct | Class         | No. of Samples | Classified into Class |     |     | % Correct |
|                   |                | 1                     | 2   | 3   |           |               |                | 1                     | 2   | 3   |           |
| 1                 | 484            | 482                   | 2   | 0   | 99.6      | 1             | 483            | 478                   | 3   | 2   | 98.9      |
| 2                 | 933            | 0                     | 885 | 48  | 94.9      | 2             | 932            | 0                     | 880 | 52  | 94.4      |
| 3                 | 483            | 0                     | 19  | 464 | 96.1      | 3             | 482            | 0                     | 16  | 466 | 96.7      |

# Derivation of the Bayes Classifier

---

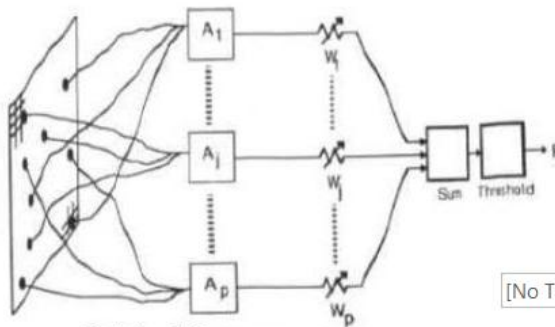




## **Classification based on Neural Networks and Deep Learning**

# The Perceptron

## Perceptron (1957)



Original Perceptron

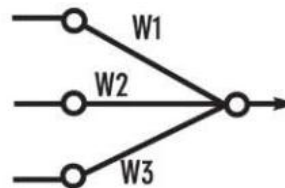
*(From Perceptrons by M. I. Minsky and S. Papert, 1969, Cambridge, MA: MIT Press. Copyright 1969 by MIT Press.)*



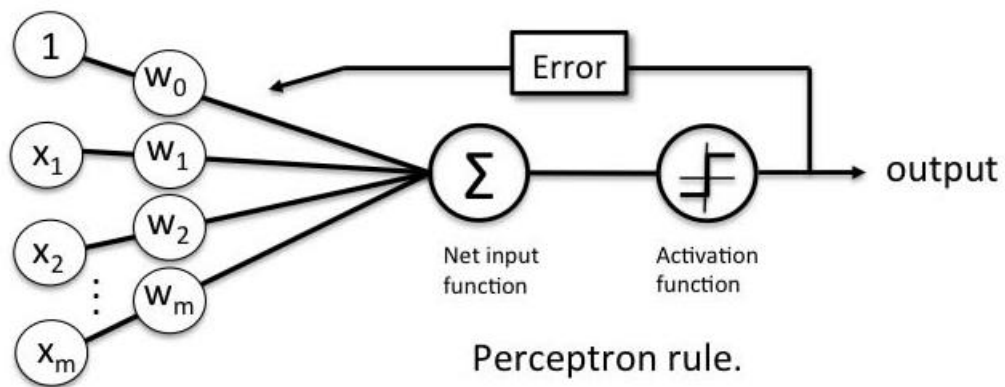
Frank Rosenblatt  
(1928-1971)

[No Title]

Simplified model:



# The Perceptron



# The Perceptron

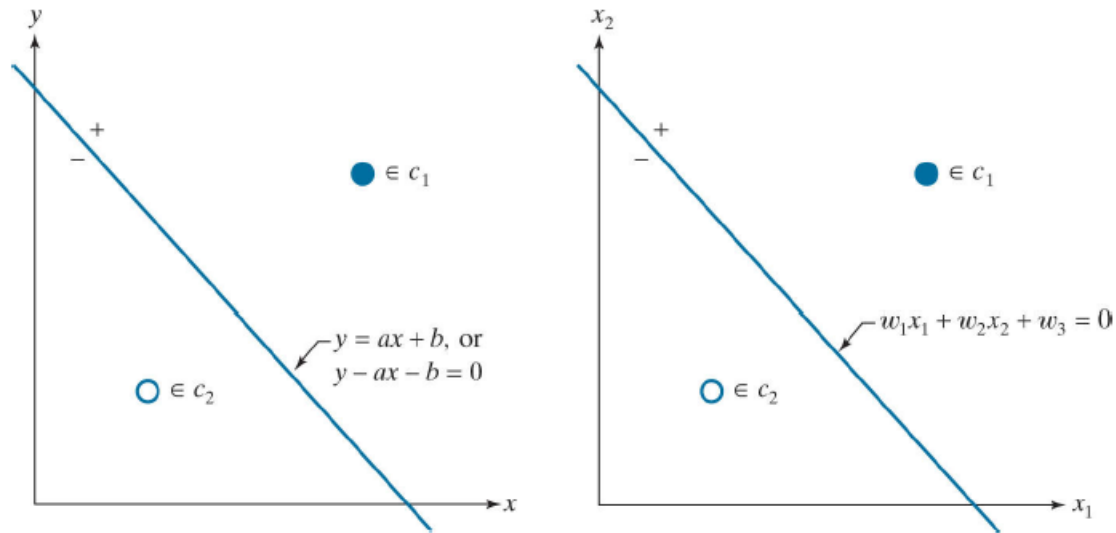
---

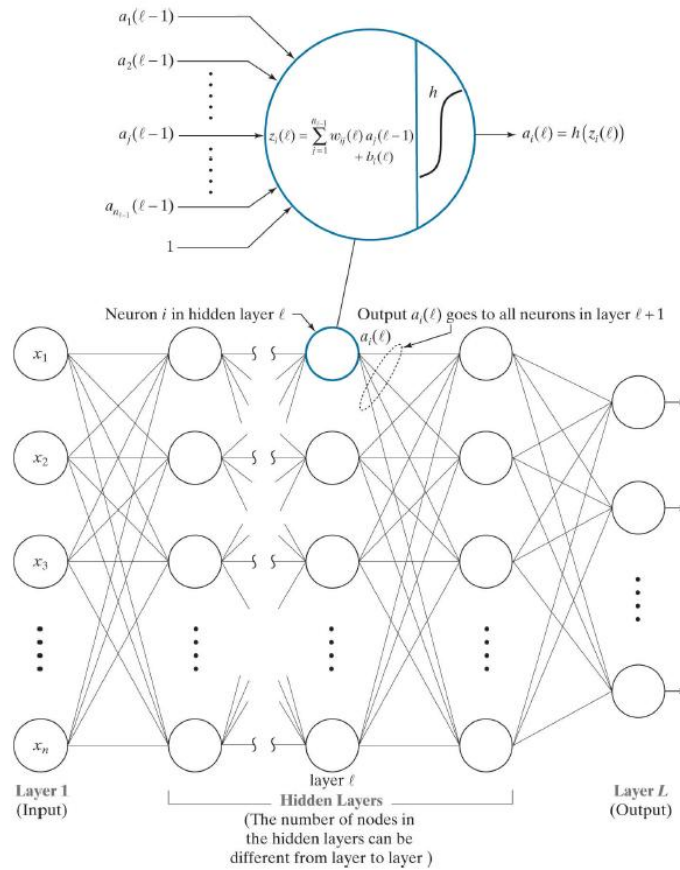
$$f(x) = \begin{cases} 1 & \text{if } w \cdot x + b > 0 \\ 0 & \text{otherwise} \end{cases}$$



$$\mathbf{w}^T \mathbf{x} = \begin{cases} > 0 & \text{if } \mathbf{x} \in c_1 \\ < 0 & \text{if } \mathbf{x} \in c_2 \end{cases}$$

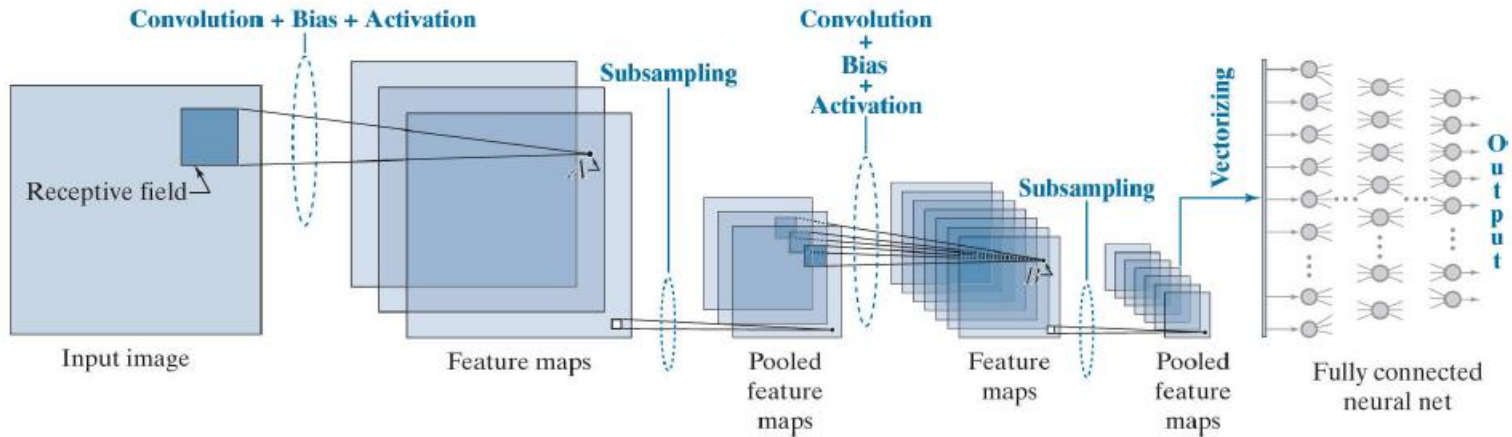
# The Perceptron



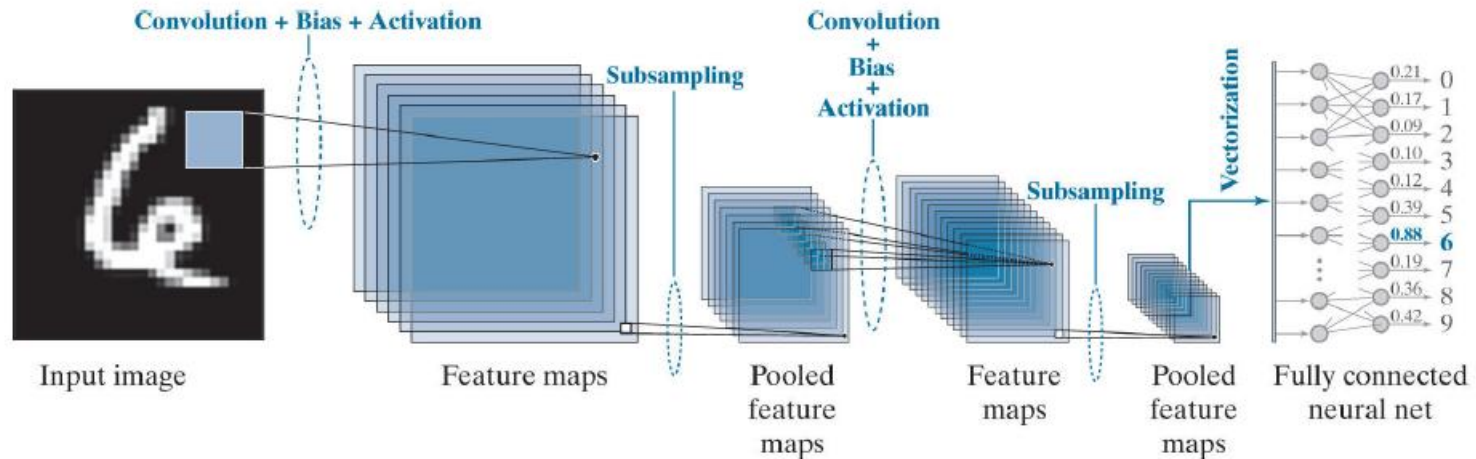


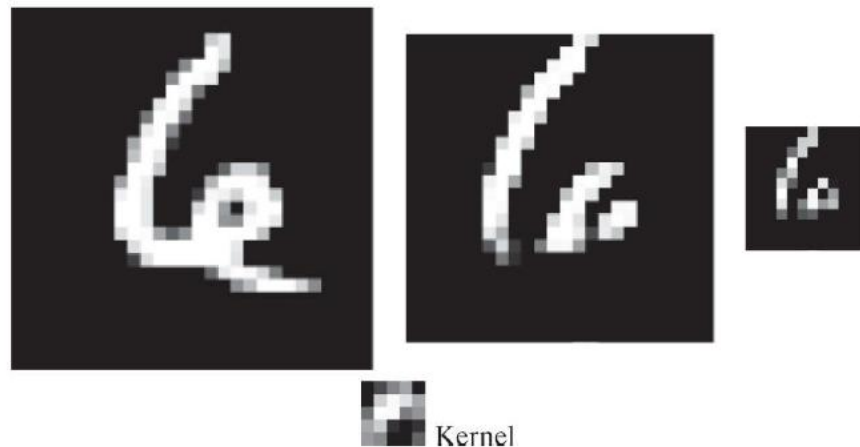
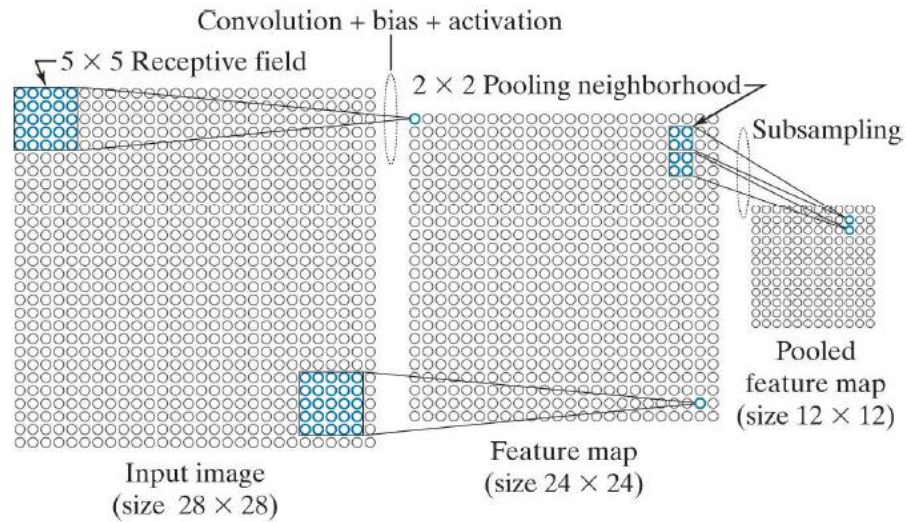


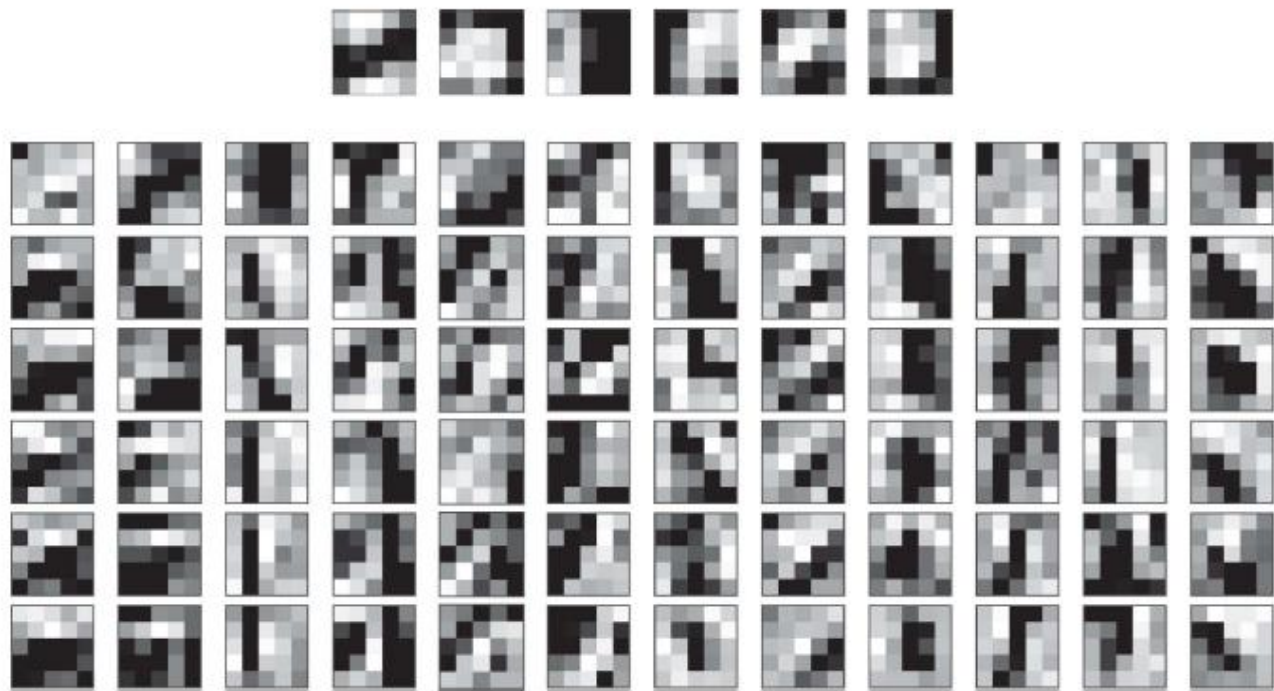
# Convolutional neural network (CNN)



# Convolutional neural network (CNN)







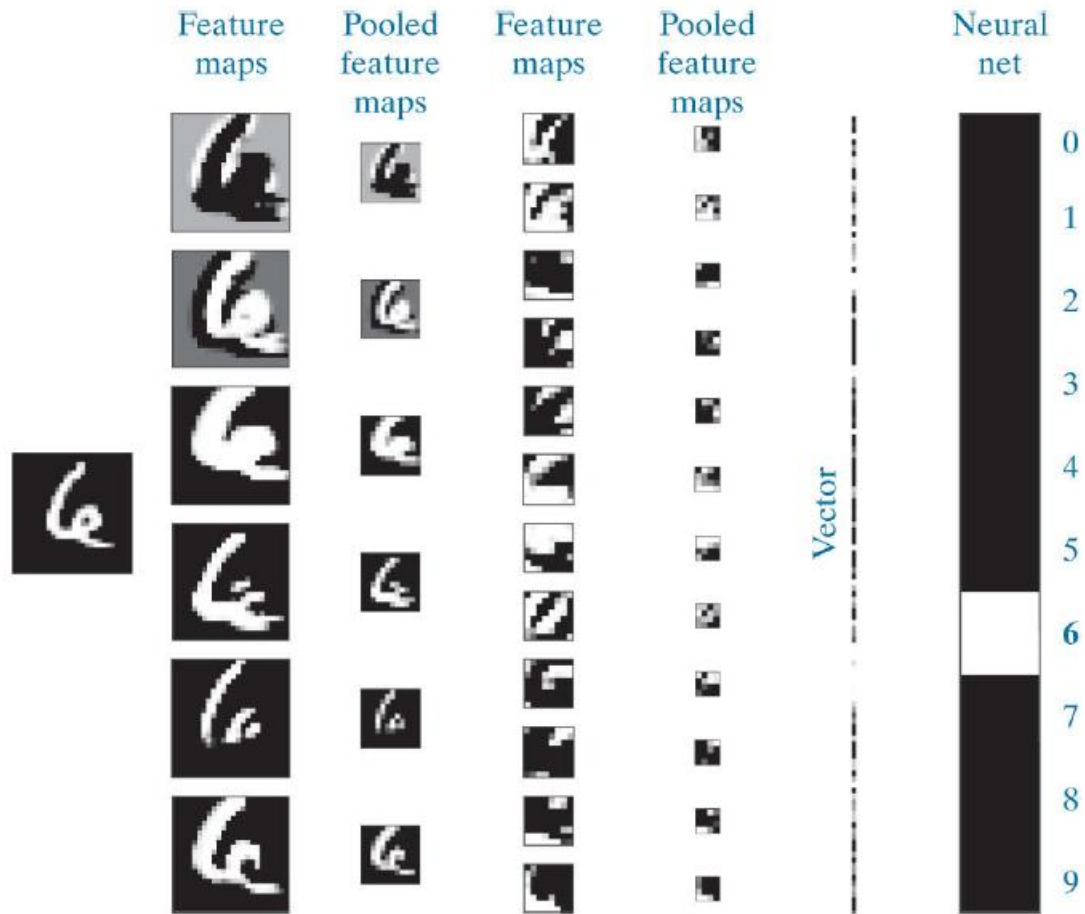
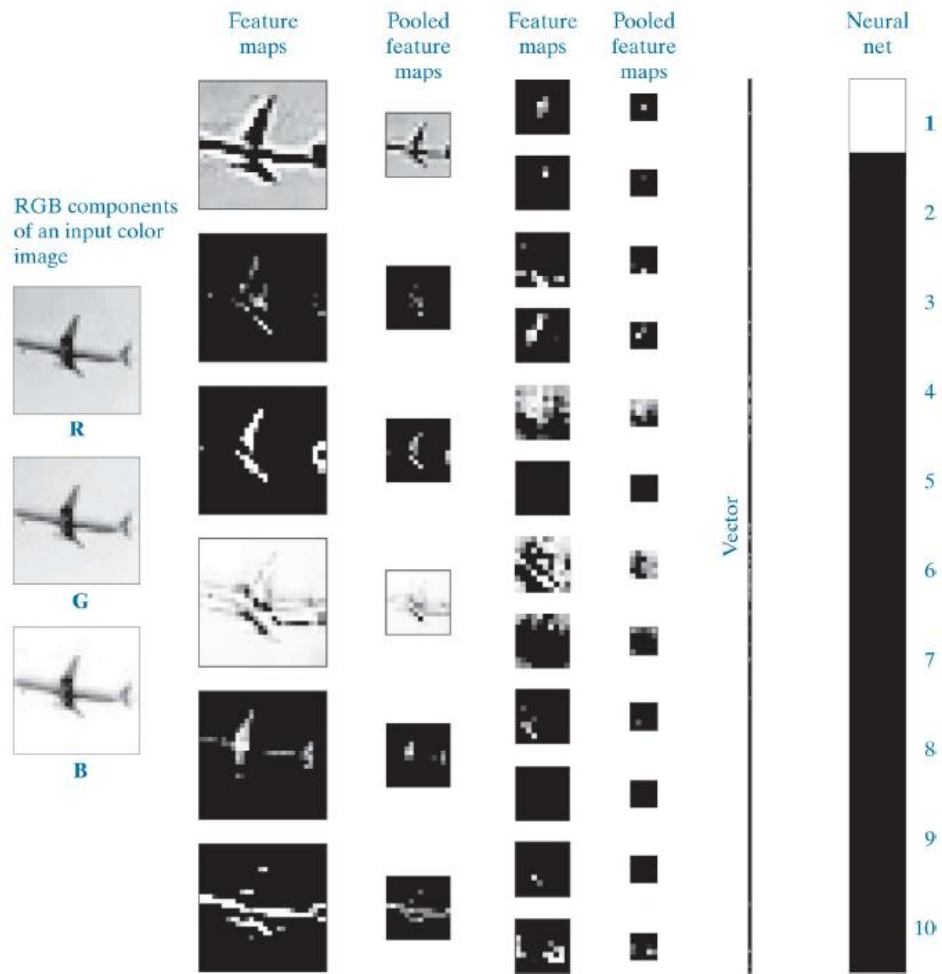








Image from: Gonzalez, R., & Woods, R. (2014). Digital image processing (3rd ed.). New Delhi: Dorling Kindersley.



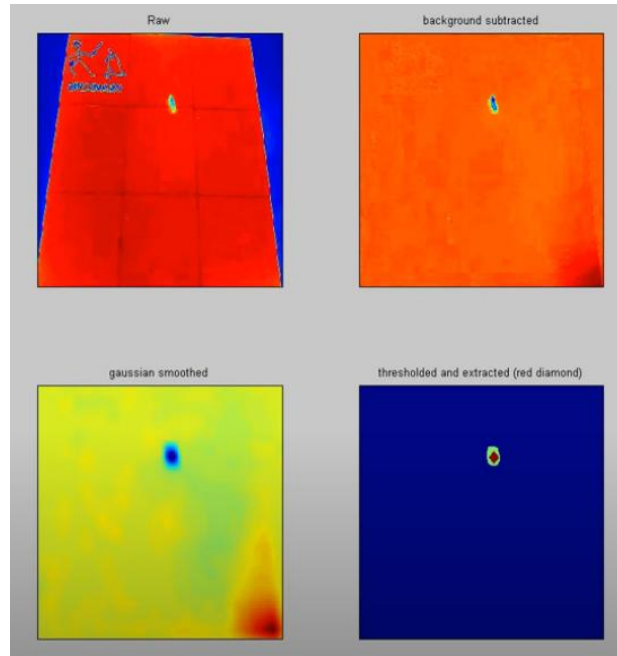




**WHAT ARE THE NEXT STEPS?**

# Object tracking

---



# Computer Vision

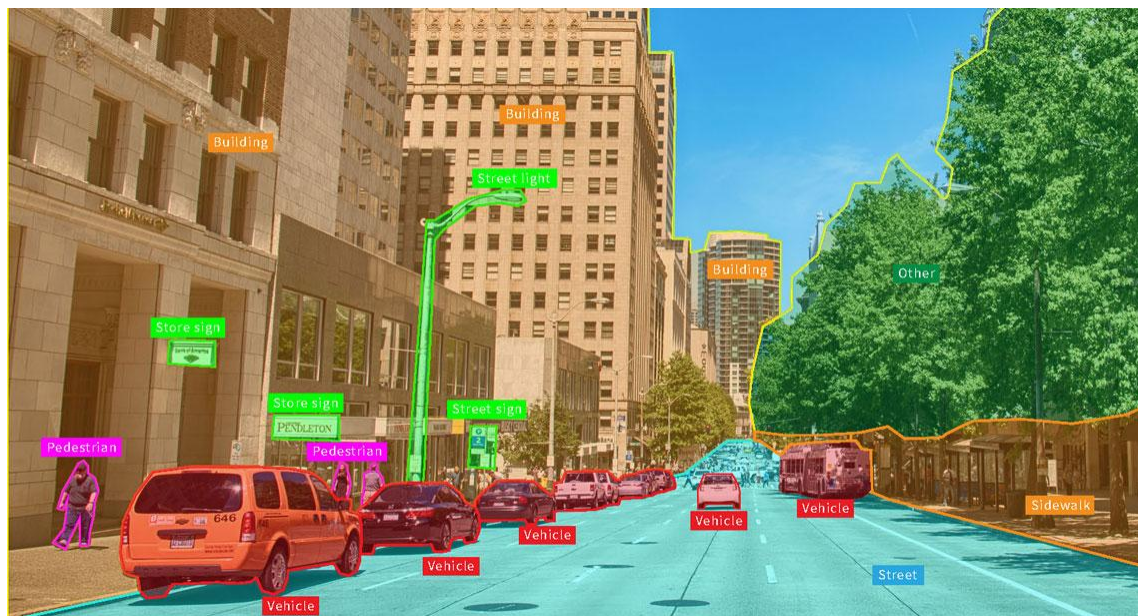
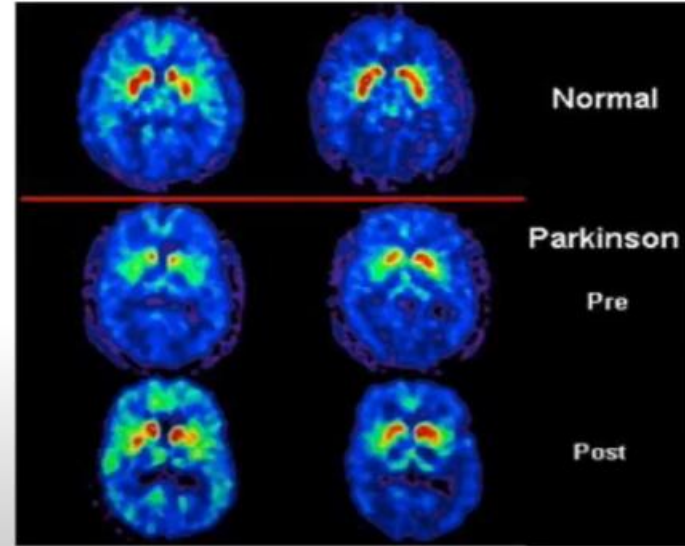
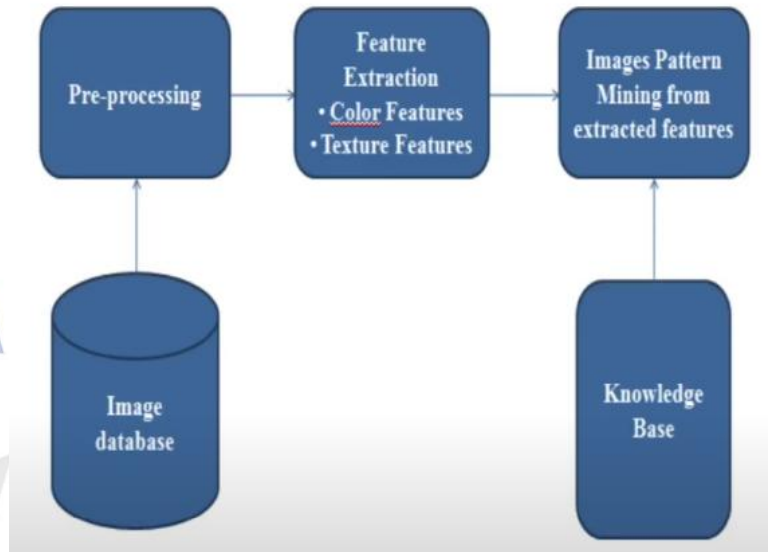


Image from: [https://cdn.appenresources.com/wp-content/uploads/2019/04/SLIDER-Appen\\_image\\_annotation\\_05.jpg](https://cdn.appenresources.com/wp-content/uploads/2019/04/SLIDER-Appen_image_annotation_05.jpg)

# Image Mining





**End of Week 13**